Python Packages

What is a Package?

- A **package** in Python is a **collection of modules** (Python files).
- A module = one .py file, while a package = **a folder** containing multiple .py files.
- Packages help in **organizing related modules** together, especially in large projects.
- To make a folder a package, it usually contains an __init__.py file (even empty).

✅ Example 1: Importing Modules from a Single Package

📂 Project Structure

```
pack1/
 ├── module1.py   → display()
 ├── module2.py   → show()
 └── client.py
```

module1.py

```
def display():
    print("This is display function from module1")
```

module2.py

```
def show():
    print("This is show function from module2")
```

client.py

```
import sys
sys.path.append("C:/Automation/PWPython/pack1")

# Approach 1: Import whole modules
    import module1, module2
    module1.display()
    module2.show()
```

Approach 2: Import functions directly

```
from module1 import *
from module2 import *
display()
show()
```

Key Points:

- sys.path.append(path) tells Python **where to find the package**.
- You can either **import whole modules** or **specific functions**.

✅ **Example 2: Importing from Two Different Packages**

📂 Project Structure

```
pack1/
 ├── module1.py   → display()
 └── pack2/
      └── module2.py  → show()
```

client.py

```
import sys
sys.path.append("C:/Automation/PWPython/pack1") # Import from pack1
from module1 import *
sys.path.append("C:/Automation/PWPython/pack1/pack2") # Import from pack2 inside pack1
from module2 import *
display()
show()
```

Key Points:

- We can import modules from **nested packages (sub-packages)**.
- Each sub-package acts like a mini folder with its own modules.

✅ **Example 3: Importing Classes from Different Packages**

📂 Project Structure

```
pack1/
 └── emp.py    → Employee class
pack2/
 └── stu.py    → Student class
pack3/
 └── client.py
```

emp.py

```
class Employee:
    def __init__(self, eid, ename, sal):
        self.eid = eid
        self.ename = ename
        self.sal = sal

    def displayemp(self):
        print(f"empid:{self.eid} empname:{self.ename} empsal:{self.sal}")
```

stu.py

```
class Student:
    def __init__(self, sid, sname, sgrad):
        self.sid = sid
        self.sname = sname
        self.sgrad = sgrad

    def displaystu(self):
        print(f"stuid:{self.sid} stuname:{self.sname} stusal:{self.sgrad}")
```

client.py

```
import sys

# Import Employee class
sys.path.append("C:/Automation/PWPython/pack1")
from emp import Employee

e = Employee(101, 'Scott', 40000)
e.displayemp()

# Import Student class
sys.path.append("C:/Automation/PWPython/pack2")
from stu import Student

s = Student(141, 'David', 'A')
s.displaystu()
```

Key Points:

- Classes can be **imported from different packages** into one client program.
- You can use sys.path.append() if the packages are not in the default Python path.
- This way, large projects remain **well-structured and modular**.

Summary

1. **Module** = a single .py file.
2. **Package** = a folder containing multiple modules.
3. You can import:
    - Whole modules (import module)
    - Specific functions/classes (from module import func/class)
    - Everything (from module import *, not recommended).
4. sys.path.append("path") → helps Python locate packages stored in custom folders.